

Lyapunov Exponents as Predictability Diagnostics for Differentiable Simulators and Learned World Models

Benjamin Strittmatter

October 2026 revision

Abstract

Draft in progress.

Contents

1	Introduction	2
2	Background	2
2.1	Lyapunov exponents and λ_1	2
2.2	Jacobians and differentiable simulation	3
2.3	The systems	4
3	Methods — what we simulate and how	5
3.1	Differentiable Warp simulators	5
3.2	The Lyapunov estimator and its calibration	6
3.3	Two estimators of λ_1	6
4	Part A — λ_1 bounds differentiable-simulator gradients	7
4.1	The gradient-gain growth law	7
4.2	The law holds across regimes	10
4.3	The gradient predictability horizon	10
4.4	Consequence: optimisation through the simulator	11
5	Part B — Lyapunov fidelity of learned world models	12
5.1	The learned world model and its prediction drift	13
5.2	Learned versus true λ_1	14
5.3	Why next-step error does not certify sensitivity	15
5.4	Structure-preserving world models	15
6	Extensions	17
7	Limitations	18
A	The Benettin/QR estimator: telescoping and the log-sum	18

Revision status. The current evidence is the corrected-simulator regime sweep (Section 4.2), trajectory optimisation (Section 4.4), and the saved five-seed learned-model study (Section 5.4). The acrobot now uses canonical momenta and implicit midpoint integration. The other retained plots (source files `fig1`, `fig3`, `fig4`, `fig6`, and `fig8`) are explicitly historical: their acrobot results use the retired velocity-state, explicit integrator and do not validate the corrected simulator. The Introduction and Abstract remain unfinished.

1 Introduction

[To be written last.]

2 Background

2.1 Lyapunov exponents and λ_1

Chaos is a concept borrowed from information theory, which quantifies how inherently difficult it is to make predictions about the future of dynamical systems. Mathematically speaking, a hallmark of chaos is the exponential divergence of nearby trajectories:

$$x(t) - \tilde{x}(t) \sim e^{\lambda t}, \quad (1)$$

where $\tilde{x}(t) = x(t) + \delta(t)$ is related to the original trajectory by a small perturbation $\delta(t)$. The exponential rate of growth is called the "Lyapunov exponent". Goal of this work is to demonstrate that the Lyapunov exponent plays an important role in characterising the predictability horizons in robotic learning.

To obtain a language that is more closely related to the robotics framework we want to study, it is advantageous to work in the tangent space picture. In practice, the trajectory $x(t)$ is determined by an ODE $\dot{x} = f(x)$, for some function $f(x(t))$. We can linearise the ODE for a perturbation $x + \delta$, and arrive at the expression

$$\dot{\delta} = Df(x) \delta, \quad (2)$$

with $Df(x)$ the Jacobian matrix. Mirroring the mechanics of a differentiable simulator, we discretise the flow with a fixed step dt . This gives a one-step map $x_{t+1} = F(x_t)$, whose linearisation propagates the perturbation by the one-step Jacobian J_t :

$$\delta_{t+1} = J_t \delta_t, \quad J_t = \frac{\partial x_{t+1}}{\partial x_t}. \quad (3)$$

Solving the discrete recursion gives $\delta_t = M_t \delta_0$, propagated by the *rollout Jacobian*

$$M_t = \frac{\partial x_t}{\partial x_0} = \prod_{s < t} J_s, \quad (4)$$

the fundamental matrix of the linearised flow. The *largest Lyapunov exponent* is the asymptotic growth rate of a generic infinitesimal perturbation,

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|M_t \delta_0\|}{\|\delta_0\|}, \quad (5)$$

a limit that exists for almost every initial condition by Oseledets’ multiplicative ergodic theorem [1]. The order of limits matters— δ_0 infinitesimal first, then $t \rightarrow \infty$ —which is why the exponent is a property of the linearised flow rather than of two finite trajectories, whose separation saturates once it is no longer small. A positive λ_1 is the operational signature of chaos: an initial uncertainty ε grows to order unity within the *predictability* (or *Lyapunov*) *time* $\tau \sim \lambda_1^{-1} \ln(1/\varepsilon)$, so $1/\lambda_1$ sets the horizon beyond which the trajectory is effectively unpredictable. A central claim of this work is that the *same* λ_1 governs the predictability horizon of both computational objects of modern robot learning—differentiable simulators (Part A) and learned world models (Part B).

The perturbation directions that grow fastest are the leading singular vectors of M_t ; its singular values $\sigma_i(M_t) \sim e^{\lambda_i t}$ define the full *Lyapunov spectrum* $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. (Singular rather than eigenvalues: M_t is non-normal, and a pure shear has unit eigenvalues yet still stretches space.) The sum of the spectrum measures the mean exponential growth of phase-space volume [2],

$$\sum_{i=1}^n \lambda_i = \langle \nabla \cdot f \rangle = \frac{1}{dt} \langle \ln |\det J| \rangle, \quad (6)$$

the time-averaged divergence of the flow. The systems we study are Hamiltonian, so Liouville’s theorem gives $\nabla \cdot f = 0$ and the spectrum sums to zero: phase-space volume is conserved. This volume-preservation constraint is a structural fingerprint that a faithful model must reproduce—a theme we return to in Part B (Section 5)—and serves throughout as a consistency check on the estimator.

Reading the *full spectrum* off an explicitly formed M_t is numerically hopeless: the dynamic range $\sigma_1/\sigma_n \sim e^{(\lambda_1 - \lambda_n)t}$ over- or under-flows and every column collapses onto the dominant direction, burying the sub-dominant exponents within a few thousand steps. We instead use the standard Benettin/QR estimator [3, 4]: evolve an orthonormal frame Q under the one-step Jacobians, re-orthonormalise at each step through a factorisation $J_t Q = Q' R$, and accumulate the logarithms of the diagonal entries of R ; the running averages converge to the exponents λ_i (reported per unit time, after dividing by dt), with convergence guaranteed by Oseledets. A single such estimator is used throughout, calibrated against known answers: it returns zero to machine precision on a harmonic oscillator and recovers $\lambda_1 \approx 0.906$ for the Lorenz system, as detailed in Section 3.2.

2.2 Jacobians and differentiable simulation

A *differentiable simulator* is a physics engine whose one-step map $x_{t+1} = F(x_t)$ is implemented so that derivatives propagate through it: applying reverse-mode automatic differentiation to a rollout returns the gradient of a trajectory objective with respect to the initial state, the physical parameters, or a sequence of control inputs. We use NVIDIA Warp [5] for every simulator in this work; the implementation details are deferred to Section 3.1.

The object such a simulator differentiates is exactly the rollout Jacobian of Section 2.1. For a scalar loss $L(x_T)$ evaluated at the end of a T -step rollout, the chain rule gives

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_T} M_T, \quad M_T = \prod_{t < T} J_t, \quad (7)$$

with M_T the rollout Jacobian of Eq. (4). Reverse-mode differentiation evaluates this product right-to-left, multiplying in one J_t per step as it walks back along the stored trajectory. Every gradient a differentiable simulator produces is therefore a product of one-step Jacobians—the same product that governs the growth of perturbations.

This is not a peculiarity of one loss. The three canonical uses of a differentiable simulator all backpropagate through the rollout and so all carry the factor M_T : system identification differentiates the trajectory with respect to physical parameters, trajectory optimisation differentiates it with respect to a control sequence, and first-order (analytic) policy gradients differentiate it with respect to policy parameters [6, 7]. In each case the gradient inherits whatever M_T does.

For a generic tangent direction, $\|M_T\| \sim e^{\lambda_1 T}$ up to subexponential factors under the assumptions of Section 2.1. This creates sensitivity to perturbations and can complicate gradient-based optimisation [8, 7, 6]. It does not, by itself, prove an optimisation cutoff at $1/\lambda_1$: objective, perturbation scale, precision, and optimiser budget matter. Part A tests the growth-rate relation and a finite-budget optimisation comparison separately.

Equation (7) also gives a second, independent handle on λ_1 . By Eq. (5), the growth rate of the gradient gain $\|M_T\|$ with T is itself a finite-horizon estimate of λ_1 —distinct from the Benettin/QR estimator of Section 2.1, computed from the same orbit, and expected to agree with it as $T \rightarrow \infty$ by Oseledets. Section 3.3 makes this comparison the basis of Part A.

2.3 The systems

We study three systems that span the range of dynamical behaviour we care about: one integrable and two chaotic of contrasting character. Each is a conservative Hamiltonian system: writing the state as a configuration q with conjugate momentum p , the dynamics follow from a Hamiltonian

$$H(q, p) = \frac{1}{2} p^\top M(q)^{-1} p + V(q), \quad (8)$$

the sum of kinetic and potential energy, with $M(q)$ the configuration-dependent mass (inertia) matrix [9]. Equivalently, in second-order form the passive dynamics obey the manipulator equation $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = 0$ [10]; control enters as a forcing term on the right, but our Lyapunov diagnostics concern the underlying conservative flow, for which Liouville’s theorem applies and the spectrum sums to zero as in Section 2.1.

The three systems form a deliberate contrast in λ_1 :

- **Pendulum** — a single degree of freedom with separable Hamiltonian $H = \frac{1}{2}p^2 + V(\theta)$. A one-degree-of-freedom autonomous Hamiltonian is integrable: energy conservation confines every orbit to a level set of H , leaving no room for exponential divergence, so $\lambda_1 = 0$ [9]. This is our ordered baseline.
- **Acrobot** — a two-link pendulum, used here as a model of chaotic underactuated dynamics [11]. Its inertia matrix $M(q)$ depends on the configuration, making the Hamiltonian *non-separable*, and its passive dynamics admit chaotic regimes, $\lambda_1 > 0$.
- **Hénon–Heiles** — a unit-mass particle in the smooth two-dimensional potential $V(x, y) = \frac{1}{2}(x^2 + y^2) + x^2y - \frac{1}{3}y^3$, a classical model from galactic dynamics [12]. Its Hamiltonian is *separable* (constant mass matrix $M = I$), yet for energies approaching the escape value $E = \frac{1}{6}$ the motion is strongly chaotic, $\lambda_1 > 0$. It is a second, mechanistically independent chaotic system—of different origin and structure from the acrobot—included to show the gradient diagnostics track chaos itself, not the mechanical setting. Its phase space is *mixed* (regular tori coexist with a chaotic sea), so we launch orbits in the chaotic sea; λ_1 is $O(0.1)$, small but genuinely positive.

The separable/non-separable distinction is the structural divide. When M is constant the kinetic term in Eq. (8) decouples from q , and the position and momentum updates can be taken independently; when $M(q)$ varies—as for the acrobot—the two are coupled. That coupling makes the acrobot the demanding case for the structure-preserving integrators and world models of Part B. Chaos does not require it, however: Hénon–Heiles is separable ($M = I$) yet chaotic, its sensitivity arising from the anharmonic potential rather than a configuration-dependent inertia.

The separable pendulum and Hénon–Heiles use symplectic Euler. For the non-separable acrobot we use canonical implicit midpoint, a symplectic method when solved exactly [13, 14]. Our implementation uses six fixed Picard iterations; its finite-precision map is checked against the analytic midpoint Jacobian. The retired explicit acrobot scheme was not symplectic and is retained only for historical comparisons.

3 Methods — what we simulate and how

3.1 Differentiable Warp simulators

We implement the systems directly in Warp [5]. The pendulum state is (θ, ω) and Hénon–Heiles uses (x, y, p_x, p_y) ; with their unit masses, a momentum-first symplectic Euler step advances

$$p_{t+1} = p_t - dt \nabla V(q_t), \quad q_{t+1} = q_t + dt p_{t+1}. \quad (9)$$

The pendulum force is $-g \sin \theta$ and the Hénon–Heiles force is $-(x + 2xy), -(y + x^2 - y^2)$.

The acrobot instead uses $z = (\theta_1, \theta_2, p_1, p_2)$ with $p = M(\theta)\omega$ and

$$M(\theta) = \begin{bmatrix} (m_1 + m_2)l_1^2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) \\ m_2 l_1 l_2 \cos(\theta_1 - \theta_2) & m_2 l_2^2 \end{bmatrix}. \quad (10)$$

Its Hamiltonian is Eq. (8), with $V = -(m_1 + m_2)gl_1 \cos \theta_1 - m_2 gl_2 \cos \theta_2$. We solve $z_{t+1} = z_t + dt \mathbb{J} \nabla H((z_t + z_{t+1})/2)$ by six fixed Picard passes, where $\mathbb{J}$ is the canonical structure matrix. Control adds a generalised force u to the p_2 equation. Fixed iteration count keeps the executed map differentiable; symplecticity is exact for the implicit solution, with numerical accuracy checked for the executed map. Historical acrobot plots used angular velocities and an explicit acceleration update.

Reverse-mode differentiation requires the full trajectory to be available on the backward pass, so each rollout is stored as a $(T+1) \times \text{dim}$ array, writing one new row per step and never overwriting a cell; the adjoint then walks back along the stored states, multiplying in one one-step Jacobian J_t per step exactly as in Eq. (7). All simulators run on CPU, single-threaded. The integration step and physical parameters are collected in Table 1.

System	dt	Parameters
Pendulum	5×10^{-3}	$g = 9.81, \ell = 1$
Acrobot	5×10^{-4}	$m_1 = m_2 = 1, l_1 = l_2 = 1, g = 9.81$
Hénon–Heiles	10^{-2}	unit mass, standard coupling ($\lambda = 1$)

Table 1: Integration step and physical parameters of the three Warp simulators.

3.2 The Lyapunov estimator and its calibration

We operationalize the Benettin/QR estimator of Section 2.1 as follows. From a seeded random orthonormal frame $Q \in \mathbb{R}^{n \times k}$ (the k leading exponents; $k = n$ for the full spectrum), we advance the tangent frame one step at a time, re-orthonormalising through a thin QR factorisation $J_t Q = Q' R$ at *every* step, accumulating $\ln|R_{ii}|$, and dividing the running sums by the elapsed time $N dt$ to obtain the exponents in physical (per-unit-time) units, sorted in descending order (the telescoping that makes $\frac{1}{N dt} \sum_t \ln R_{ii}^{(t)} \rightarrow \lambda_i$ exact is spelled out in Appendix A). The initial transient is discarded by slicing the trajectory before the average: for the dissipative Lorenz calibration this is the relaxation onto the strange attractor (the first 2000 steps), and this trajectory slicing does not burn in the QR frame, which is initialised afterwards. Finite-window QR-frame dependence therefore remains part of the measurement. A single such estimator is used for every result in this paper.

We calibrate it against two systems with known answers.

- **Harmonic oscillator** (an exact rotation map, all exponents zero): the estimator returns $|\lambda_i| \lesssim 2 \times 10^{-15}$ over 2×10^4 steps—zero to machine precision, as expected since each step contributes $\ln|\pm 1| = 0$ exactly.
- **Lorenz** ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$; RK4 at $dt = 5 \times 10^{-3}$, first 2000 steps dropped): the finite-time largest exponent is itself a fluctuating quantity. Single-trajectory estimates scatter by ~ 0.02 around the standard literature value $\lambda_1 = 0.9056$ over a few hundred Lyapunov times, and the running average on a long trajectory converges to it ($\lambda_1 = 0.905$ at 1.6×10^3 Lyapunov times). This scatter is intrinsic finite-time variance, not a Jacobian or estimator bias: finite-difference and analytic (variational-RK4) Jacobians agree on λ_1 to 3×10^{-8} along a common orbit. As a sharp structural check valid at *every* horizon, the recovered spectrum has a near-zero middle exponent and sums to $\sum_i \lambda_i = -13.667$, matching the constant flow divergence $\text{tr } Df = -(\sigma + 1 + \beta)$ to machine precision (Eq. (6)).

The pendulum and Hénon–Heiles use analytic one-step Jacobians. The corrected acrobot uses the midpoint Cayley Jacobian $(I - dt A/2)^{-1}(I + dt A/2)$, with $A = \mathbb{J} \nabla^2 H$ at the computed midpoint. Tests cross-check these against Warp autodiff and check the acrobot’s fixed-point and volume-preservation accuracy in the operating envelope. The analytic formula represents the implicit map; agreement with the finite-iteration implementation must be verified, rather than assumed.

3.3 Two estimators of λ_1

Part A rests on comparing two finite-horizon estimates of λ_1 computed from the *same* orbit. The first is the calibrated Benettin/QR estimator of Section 3.2. The second reads λ_1 off the growth of the gradient gain introduced in Section 2.2: since $\|M_T\| \sim e^{\lambda_1 T}$, the least-squares slope of $\ln \|M_T\|_2$ against T is itself an estimate of λ_1 . Here $\|M_T\|_2$ is the spectral norm (top singular value) of the rollout Jacobian, and the slope is fit over a range of horizons reaching several Lyapunov times (T from $\sim 10^3$ up to 1.6×10^4 integration steps).

These two numbers are distinct finite- T functionals of the same Jacobian sequence, *not* algebraically identical: the QR estimator averages the local logarithmic stretching of an orthonormal frame, whereas the slope tracks the global spectral norm of the accumulated product. Both converge to the true λ_1 as $T \rightarrow \infty$ by Oseledets’ theorem, but only once $T \gg 1/(\lambda_1 - \lambda_2)$ —the time a generic direction needs to align with the leading one—and the slope is the higher-variance of the two. Part A therefore reports their agreement as a *consistency relation*—evidence that the gradient-gain growth rate and the dynamical λ_1 are one and the same number—rather than as an exact identity.

4 Part A — λ_1 bounds differentiable-simulator gradients

Part A establishes the first pillar of the thesis: the largest Lyapunov exponent λ_1 governs the gradients a differentiable simulator produces. Section 2.2 showed that every such gradient carries the rollout-Jacobian factor M_T with $\|M_T\| \sim e^{\lambda_1 T}$, and Section 3.3 set up a second, gradient-side estimate of λ_1 read off that growth rate.

Three systems play distinct roles. The chaotic *acrobot* and *Hénon–Heiles* are the workhorses and carry the λ_1 claims; the integrable *pendulum* ($\lambda_1 = 0$) is the null contrast, whose gradient grows only linearly, never exponentially. Hénon–Heiles, with a modest $\lambda_1 \approx 0.13$ close in magnitude to the pendulum’s finite-time floor, is the sharpest test that the diagnostics track chaos itself rather than merely a large exponent.

The current regime sweep tests agreement between the gradient-gain slope and QR estimate. The optimisation experiment tests how a precise objective behaves as the horizon grows under a fixed budget. Earlier growth-law and gradient-SNR plots are retained below as historical context, not current acrobot evidence.

4.1 The gradient-gain growth law

Historical evidence. These two plots retain the earlier velocity-state acrobot experiment and its explicit, non-symplectic integrator. The old acrobot growth rates (1.29 versus 1.28 s^{-1}) and apparent convergence are superseded as evidence for the corrected simulator. They illustrate the original diagnostic; Section 4.2 supplies the current comparison.

The measured gain is $\|M_T\|_2$, so its asymptotic exponential rate is λ_1 , not $2\lambda_1$. Squaring the gain doubles that rate. Regular anharmonic motion can instead produce polynomial tangent growth; a positive finite-window estimate alone does not establish a positive asymptotic exponent.

Rollout-Jacobian gain $\|M_T\|_2$: linear (integrable) vs exponential (chaotic)

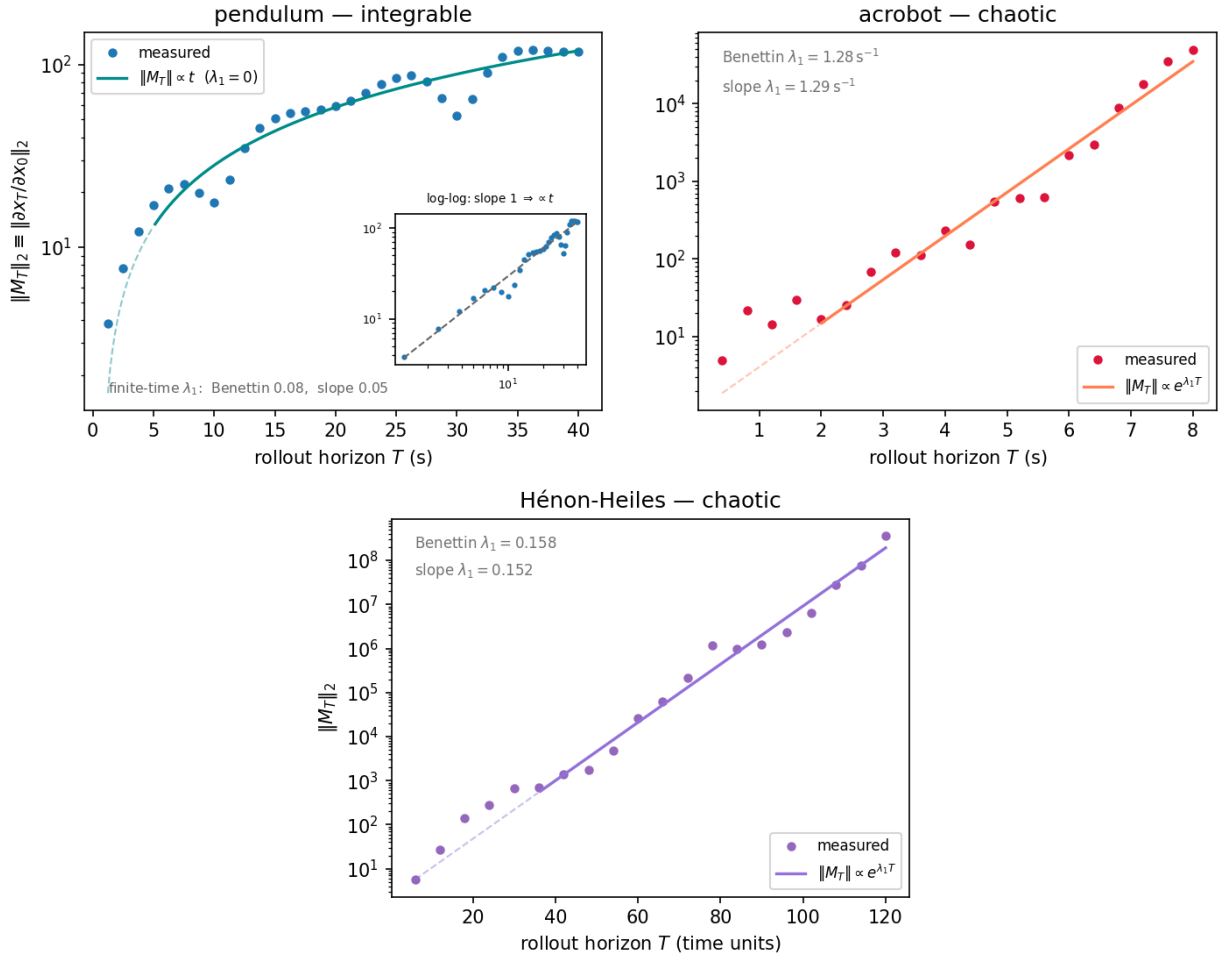


Figure 1: **Historical, pre-correction.** Gradient gain versus horizon under the older protocol. The acrobot panel uses the retired velocity-state integrator; its fitted rates do not substantiate the current simulator.

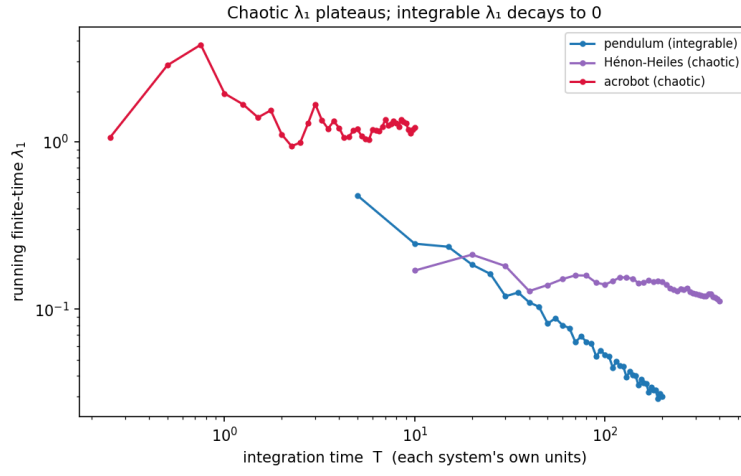


Figure 2: **Historical, pre-correction.** Running QR estimates under the older protocol. The apparent acrobot plateau belongs to the retired integrator; no corrected convergence claim is made from this plot.

4.2 The law holds across regimes

We sweep six canonical acrobot initial states $(\theta_1, 0, 0, 0)$ with $\theta_1 \in \{0.5, 1, 1.5, 2, 2.5, 3\}$, one pendulum state $(2, 0)$, and Hénon–Heiles chaotic-sea initial conditions at energies 0.11, 0.13, 0.145, 0.16. The existing protocol compares gradient-gain slopes and same-orbit QR estimates at horizons 1, 2, $\dots$, 8 seconds for the acrobot, 10, 20, $\dots$, 80 seconds for the pendulum, and 20, 40, $\dots$, 120 time units for Hénon–Heiles. The corrected acrobot uses $dt = 0.0005$ seconds. Across these 11 points the correlation is 0.969, with regression slope 0.986 (ideal 1) and intercept -0.001 . The acrobot QR estimates span $0.080\text{--}1.161\text{ s}^{-1}$ non-monotonically with initial energy. The result supports approximate agreement across these regimes, with visible finite-window deviations; it is not equality at every point.

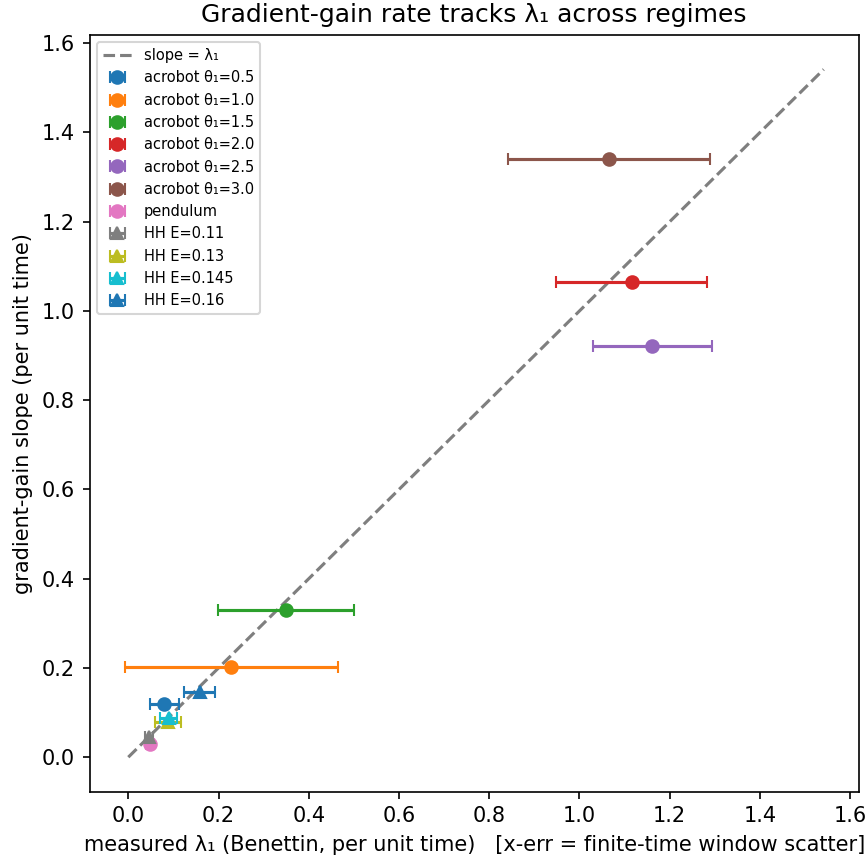


Figure 3: Current corrected-simulator regime sweep: gradient-gain slope versus same-orbit Benettin λ_1 for six acrobot states, one pendulum state, and four Hénon–Heiles energies. Correlation 0.969, regression slope 0.986. Each system uses its own inverse-time units; $y = x$ expresses equality of the two rates. Horizontal bars show variation across finite-time windows, not confidence intervals.

4.3 The gradient predictability horizon

Historical evidence. The earlier experiment compared the mean analytic gradient of $\|x_T\|^2$ to its variation across nearby initial conditions. Its acrobot curve used the retired velocity-state integrator. The reported crossover near one Lyapunov time has not been re-established under the corrected simulator. Gradient SNR across an ensemble and deterministic trajectory optimisation are different questions: the former depends on the perturbation ensemble, the latter on objective, precision and

optimiser budget. The current optimisation comparison follows below.

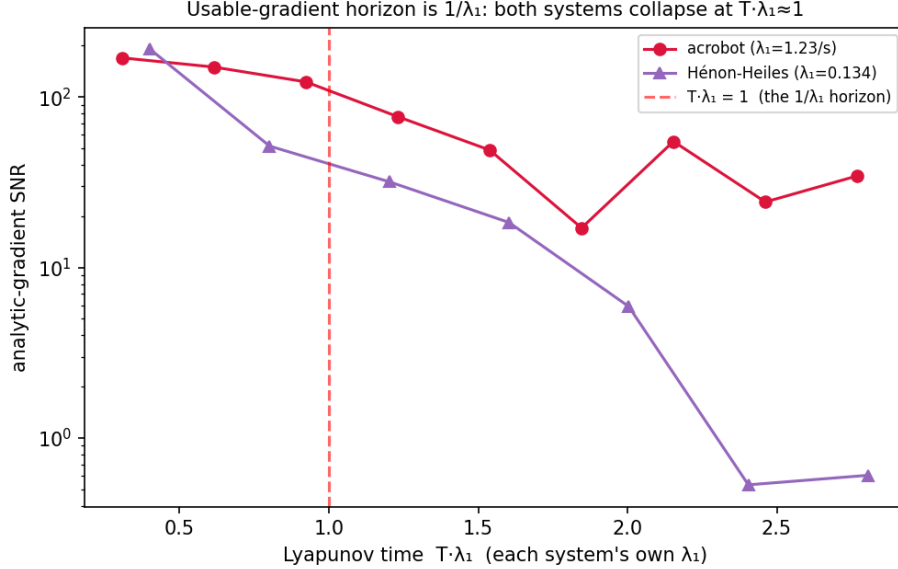


Figure 4: **Historical, pre-correction.** Gradient SNR across perturbed initial states under the older protocol. The acrobot curve uses the retired velocity-state integrator. This plot is not current evidence for a universal optimisation cutoff at one Lyapunov time.

4.4 Consequence: optimisation through the simulator

We compare two objectives on the corrected canonical acrobot: reaching a specific feasible terminal state and moving both links upright. An integrable pendulum provides a precise-reaching control. The acrobot input is a generalised force on p_2 . Each precise target is generated by a known random control sequence, so a zero-error solution exists, but a finite-budget optimiser need not find it.

The existing protocol uses $dt = 0.0005$ seconds, canonical acrobot start $(2.5, 0, 0, 0)$ and pendulum start $(2, 0)$, with matched step counts on the acrobot’s Lyapunov axis. Its passive QR estimate is measured over 7 seconds after slicing off the first second of an 8-second rollout, with $k = 4$ and QR seed 0. It is a different finite-window reference from the 54-second study in Part B. At each of $T\lambda_1 = 1, 2, 3, 3.5, 4, 4.5, 5$, Adam runs for 250 iterations: 12 acrobot reach targets, six pendulum targets, and six swing-up initialisations. Learning rates are 0.3 for reaching and 1.0 for swing-up. Reach controls start at zero; swing-up starts from seeded Gaussian controls (standard deviation 0.3). The swing-up loss adds $10^{-7} \sum_t u_t^2$ to the terminal angle cost; the plotted cost excludes this regulariser. It requires neither balancing nor small terminal velocities. Cost is normalised by the corresponding zero-control rollout; the curve is a geometric mean and the band is the 10th–90th percentile across seeds.

The measured reference is $\lambda_1 = 1.00656 \text{ s}^{-1}$. Acrobot reaching is non-monotonic below the long-horizon deterioration: its geometric-mean cost ratio is 0.00018 at $T\lambda_1 = 1$, 0.0187 at 2, 0.000052 at 3, and 0.0497 at 3.5. It then rises above the zero-control baseline to 4.97 at 4, 51.5 at 4.5, and 44.7 at 5. The widening spread across targets matters: the 10th–90th percentile range at 4 is approximately 0.18–420. A mean curve does not imply that all targets fail at the same horizon. The pendulum reaching means remain between 2.7×10^{-9} and 3.7×10^{-8} of baseline across the

seven horizons. Swing-up means range from 9.8×10^{-7} to 0.054, also below baseline throughout, although individual starts can finish poorly. The aggregate contrast supports an objective- and budget-dependent precision loss on this chaotic trajectory.

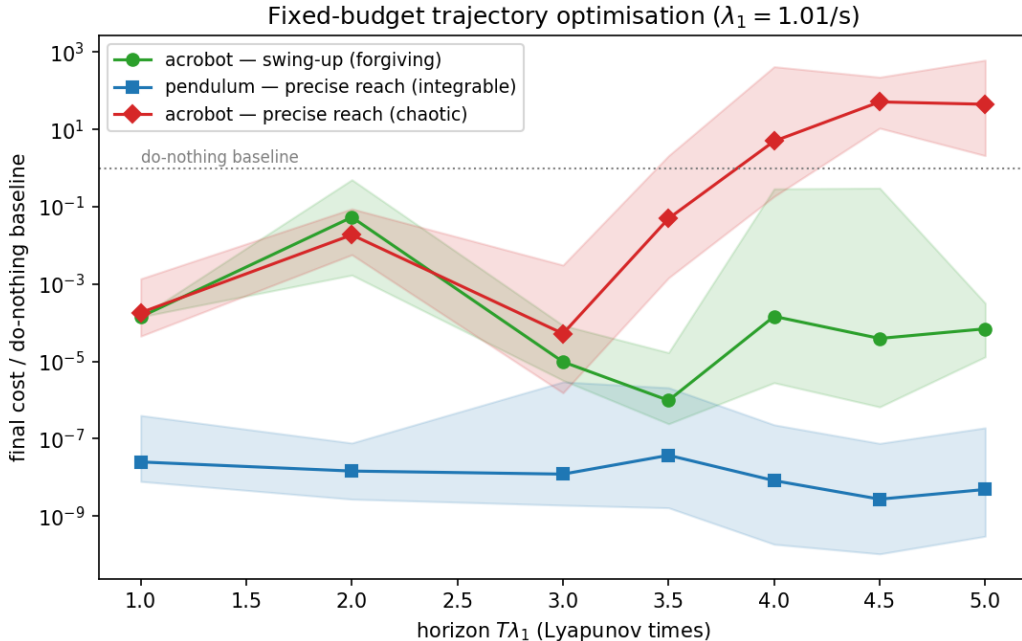


Figure 5: Current corrected-simulator comparison under a fixed 250-iteration Adam budget: normalised terminal cost versus horizon on the acrobot’s Lyapunov axis. Geometric means with 10th–90th percentile bands, across 12 acrobot reach targets, six pendulum targets, and six swing-up starts. The dotted line is the zero-control baseline; lower is better. Each objective uses its own baseline.

This measures an observed loss of precision under one optimisation protocol, not an impossibility boundary. The perturbation-growth estimate $T \sim \lambda_1^{-1} \log(1/\varepsilon)$ motivates a precision-dependent horizon, but this experiment does not fit that law or isolate sensitivity from optimisation budget and conditioning. In particular, a deterministic rollout gradient may remain useful beyond one Lyapunov time. The historical SNR experiment concerns an ensemble of perturbed starts and cannot substitute for this optimisation measurement.

5 Part B — Lyapunov fidelity of learned world models

Part B asks whether a next-step predictor also reproduces the simulator’s sensitivity. The historical single-seed comparisons below motivated this question. The current evidence is the five-training-seed study in Section 5.4: the HNN’s measured sensitivity varies less across seeds, but the selected penalty MLP has a slightly closer median. Neither exact sensitivity recovery nor a universally necessary hard constraint follows from these measurements.

5.1 The learned world model and its prediction drift

Historical protocol. This subsection reports the earlier single-seed experiment, including the retired velocity-state acrobot integrator. Its acrobot numbers are superseded by Section 5.4; they do not describe the corrected simulator.

A world model replaces the simulator’s one-step map $x_{t+1} = F(x_t)$ with a learned surrogate g_ϕ , fit so that iterating it reproduces trajectories of the true system. We deliberately use the smallest model that exhibits the effect: a residual multilayer perceptron $g_\phi(x) = x + \text{net}_\phi(x)$ with two tanh hidden layers, trained by next-step mean-squared error on rollouts of the true simulator. It is given only the state pairs (x_t, x_{t+1}) —no equations of motion, no Hamiltonian, no conserved quantity—which is exactly the regime in which one hopes a generic learner will simply absorb the dynamics from data.

Such a model can be accurate one step at a time yet useless over a rollout, so the first question is how quickly its predictions drift. We roll g_ϕ forward from a shared initial condition and track the prediction error $\|\hat{x}_t - x_t\|$ against the true trajectory (Fig. 6). On both systems the error grows, but at visibly different rates: on the integrable pendulum it rises slowly, while on the chaotic acrobot it climbs roughly twice as fast before saturating. A log-linear fit over the pre-saturation window gives effective rates of 0.96 s^{-1} (pendulum) and 1.74 s^{-1} (acrobot).

These rates measure *model fidelity*, not the Lyapunov exponent. Two effects are superimposed: the model’s own approximation error, which accumulates even on the non-chaotic pendulum, and—on the acrobot—the genuine chaotic amplification of that error. The pendulum makes the distinction concrete: its fitted 0.96 s^{-1} is nowhere near its true $\lambda_1 \approx 0$, so the drift rate cannot be read as λ_1 . Figure 6 is therefore a qualitative statement—prediction degrades faster on chaotic dynamics—and isolating the exponent the model has actually learned requires probing its Jacobian directly, the subject of Section 5.2.

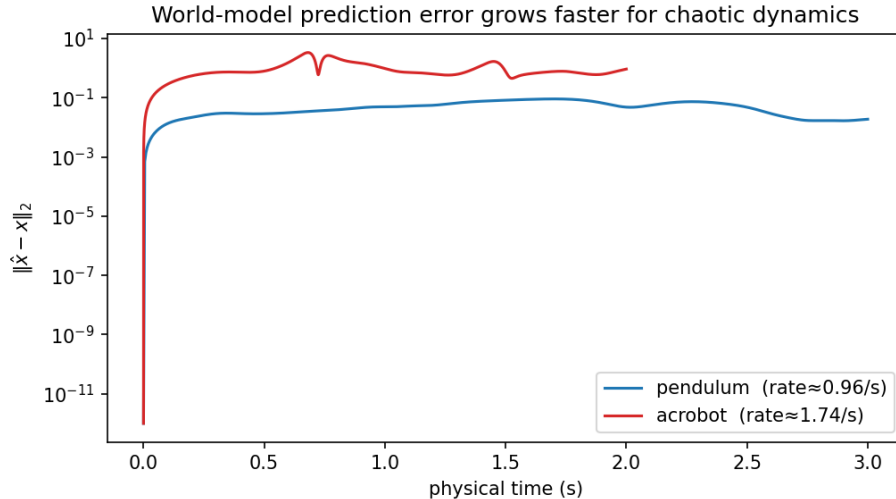


Figure 6: **Historical, pre-correction.** The acrobot uses the retired velocity-state integrator. Residual-MLP world-model prediction error $\|\hat{x}_t - x_t\|$ versus physical time (semilog). Error grows on both systems but faster on the chaotic acrobot (effective log-slope 1.74 s^{-1}) than on the integrable pendulum (0.96 s^{-1}). These are model-fidelity rates, not λ_1 : the pendulum’s is far from its true $\lambda_1 \approx 0$. The Lyapunov comparison is Section 5.2.

5.2 Learned versus true λ_1

Historical protocol. This subsection reports the earlier single-seed experiment, including the retired velocity-state acrobot integrator. Its acrobot numbers are superseded by Section 5.4; they do not describe the corrected simulator.

To read off the exponent the model has learned, we apply the *same* Benettin/QR estimator used for the true systems (Section 3.2) to the model’s one-step Jacobian $\partial g_\phi / \partial x$, obtained by automatic differentiation. The only subtlety is where to evaluate it: rolling the model out and estimating λ_1 along its *own* trajectory would conflate an inaccurate Jacobian with a drifted orbit, so we instead evaluate the learned Jacobian along the *true* trajectory, after the same transient. Estimator, orbit, and transient are then identical for the learned and reference exponents; only the source of the Jacobian differs—true physics versus learned model—so any gap is the model’s Jacobian alone. A faithful model, whose Jacobians match the true ones along the orbit, would return the true λ_1 .

Figure 7 plots learned against true λ_1 for the two systems. On the integrable pendulum the model lands on the truth: a learned $\lambda_1 \approx 0.046 \text{ s}^{-1}$ against a true 0.063 s^{-1} , both effectively zero—the network correctly reports non-chaotic dynamics. On the chaotic acrobot it does not: the learned exponent is 3.45 s^{-1} against a true 1.04 s^{-1} , an over-amplification by a factor of 3.3. The model is not merely imprecise here; it manufactures sensitivity the true dynamics do not have.

The asymmetry is the point. The network is trained to low next-step error on both systems, yet it reproduces λ_1 only where the exponent is trivial and inflates it precisely where chaotic sensitivity makes it matter. Low next-step error is therefore no guarantee of dynamical fidelity: λ_1 is a property the training objective never sees, and on chaotic dynamics the learned model overstates it threefold. Section 5.3 explains why next-step regression is blind to it.

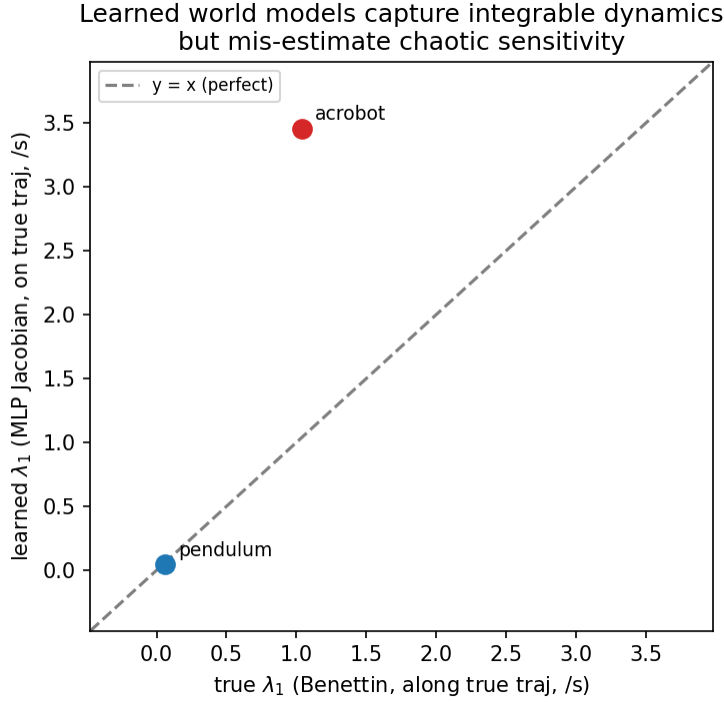


Figure 7: **Historical, pre-correction.** The acrobot uses the retired velocity-state integrator. Learned versus true λ_1 , both from the same Benettin/QR estimator evaluated along the true orbit. The integrable pendulum sits on the $y = x$ line (both $\lambda_1 \approx 0$); the chaotic acrobot lies far above it—learned 3.45 s^{-1} versus true 1.04 s^{-1} , an over-amplification of $3.3\times$. Next-step accuracy does not imply the right λ_1 .

5.3 Why next-step error does not certify sensitivity

Next-step mean-squared error supervises mapped states, not their derivatives directly. With finite data, low prediction error therefore does not by itself certify the Jacobian products used by the sensitivity diagnostic. It also does not prevent a learner from recovering good derivatives; these are separate quantities to measure. Volume preservation constrains the sum of exponents, not the largest exponent alone. A symplectic model can have the wrong λ_1 , and a nonsymplectic model can match it. The corrected study uses the same canonical coordinates for all models and the reference, avoiding finite-window coordinate changes in this comparison.

5.4 Structure-preserving world models

We compare a residual MLP, an MLP with penalty $\mu(\log |\det Dg_\phi|)^2$, and a Hamiltonian neural network (HNN). The HNN learns $H_\phi(q, p)$ with periodic angle embeddings and uses implicit midpoint; its ideal step is symplectic. Training uses midpoint Hamiltonian-gradient targets. The finite-iteration implementation and analytic midpoint Jacobian are checked separately; the structural constraint does not guarantee a correct largest exponent.

The committed `writeup/figures/partb_ablation.json` contains 40 measurements: eight conditions at training seeds 0–4, all with dataset seed 0. Each dataset contains 80 trajectories of 2000 steps. The MLPs train for 120 epochs and the HNNs for 200, with Adam (learning rate 10^{-3} ,

batch size 256). This compares the existing model protocols, not equal training-compute budgets. Each exponent averages eight QR initialisations with $k = 4$ over 54 seconds, $dt = 0.0005$ seconds, after discarding 1200 trajectory steps (0.6 seconds) from 109200 steps beginning at canonical state $(2.5, 0, 0, 0)$. All learned Jacobians are evaluated along this *reference simulator trajectory*, not autonomous learned-model rollouts. These are finite-time tangent-growth diagnostics along a common itinerary. The matching reference is 1.11634 s^{-1} ; its full QR-initialisation range is 0.07296 s^{-1} .

Condition	Median λ_1	Training-seed min-max
A: corrected data, midpoint HNN	0.9871	0.9504–1.0280
B: corrected data, retired HNN step	0.9887	0.9513–1.0264
C: retired data, midpoint HNN	0.9922	0.8717–1.0155
D: retired data, retired HNN step	0.9915	0.8742–1.0146
Plain MLP	1.2790	1.0035–1.6348
Penalty MLP, $\mu = 0.1$	1.9996	1.3741–3.3772
Penalty MLP, $\mu = 1$	2.3108	2.0562–2.5441
Penalty MLP, $\mu = 10$	1.2280	0.8889–1.5178

Table 2: Saved study, all rates in s^{-1} under the shared 54-second protocol. Ranges vary training seed only and are not confidence intervals. B reuses A’s weights; D reuses C’s weights. Retired training trajectories are mapped into canonical coordinates.

The penalty weight $\mu = 10$ was selected from $\{0.1, 1, 10\}$ by median proximity to the reference on this evaluation. It is not an independently validated optimum. Its median is slightly closer to the reference than the HNN’s, while the HNN has a much smaller observed training-seed range (0.0776 versus 0.6289 s^{-1}). This supports stability across the tested training initialisations, not a claim that only a hard constraint recovers sensitivity.

The ablation shifts are small relative to the predeclared reporting thresholds: B–A is $+0.0016$ against 0.0776 , and C–A is $+0.0051$ against 0.1438 (all s^{-1}). Each threshold is the larger full training-seed range of the two compared conditions. Neither effect exceeds its threshold, so no quantitative decomposition panel is shown. The additivity residual is 0.0023 , within the 0.1403 tolerance (the larger A/D range); this does not prove exact additivity. Below-threshold effects are not proved zero, and the remaining HNN shortfall (0.1293) is not proved exclusively model error.

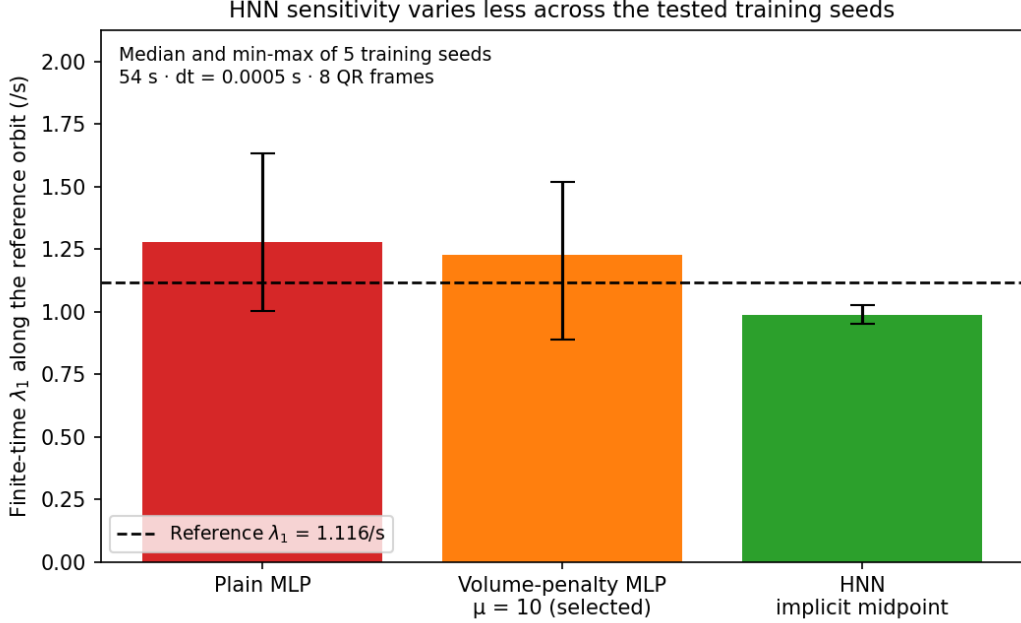


Figure 8: Saved five-seed study: median and min-max of plain MLP, selected penalty MLP ($\mu = 10$), and corrected midpoint HNN, with matching reference 1.11634 s^{-1} . Protocol: 54 seconds, $dt = 0.0005$ seconds, eight QR frames, $k = 4$, canonical $(2.5, 0, 0, 0)$; learned Jacobians follow the reference trajectory. Whiskers reflect training seeds, not confidence intervals.

6 Extensions

The diagnostics here are deliberately demonstrated on low-dimensional, single-threaded CPU systems. The natural next steps push them onto GPU hardware and onto the large learned models of the surrounding Physical-AI stack.

- **Scale to GPU.** Every diagnostic in this paper—the Benettin/QR estimator, the gradient-gain slope, and the perturbed-rollout SNR—is a sweep or ensemble average over many *independent* trajectories (initial-condition averaging, the energy sweep, the SNR ensemble). The outer loop over trajectories, seeds, and sweep points is therefore *embarrassingly parallel* and maps directly onto the GPU—in the spirit of the thousands of parallel environments used for large-scale robot learning—with only the per-trajectory time-stepping remaining sequential. A future GPU implementation could accelerate these sweeps; it is not a prerequisite for the present diagnostics or a small qualitative video-model demonstration.
- **Cosmos / large world models.** The Part-B fidelity test extends from our small surrogate to a large video world model such as **Cosmos**: does it preserve the physical Lyapunov spectrum, or hallucinate (and wash out) sensitivity? The error-growth diagnostic (Section 5.1) transfers to pixel space as a qualitative probe; the full Lyapunov-fidelity diagnostic (Section 5.2) requires recovering physical state (θ, ω) from generated video—a perception front-end—or a latent-space tangent-vector adaptation, both research problems rather than a front-end swap. These quantitative extensions are distinct from a small qualitative comparison of generated videos.
- **GR00T / action models.** For a closed-loop policy such as **GR00T**, the natural robustness diagnostic is the finite-time Lyapunov exponent of the observation $\rightarrow$ action $\rightarrow$ next-state

map, with the policy in a simulation loop (Isaac Lab / Newton) and numerical (perturbed-rollout) sensitivities, since a large policy with a finite context window is a non-Markovian, non-autodiff-friendly map.

- **Contact and stiff dynamics.** A distinct gradient-pathology regime—non-smooth rather than chaotic—deliberately outside this conservative CPU study, but the setting where differentiable-simulator gradients most often break in practice.

7 Limitations

All reported exponents are finite-window estimates. Window length, QR initialisation, coordinates, discretisation and initial state matter; this study does not establish asymptotic convergence for every condition. The five-seed model comparison holds the dataset fixed and does not measure data-sampling variation. Penalty selection reuses the evaluation, and reference-itinerary Jacobians do not establish autonomous-rollout fidelity. Volume preservation alone cannot determine the largest exponent. The optimisation comparison uses one fixed optimiser budget and does not establish a universal impossibility boundary. The explicitly historical figures use an obsolete acrobot implementation. These low-dimensional CPU results do not validate Cosmos, video world models, GPU performance, or real-robot behaviour.

A The Benettin/QR estimator: telescoping and the log-sum

Sections 2.1 and 3.2 assert that averaging the logarithms of the QR diagonals recovers the spectrum; we record here why, since the running sum is never formed explicitly in the main text.

Let $J_0, \dots, J_{N-1}$ be the one-step Jacobians along an orbit and $Q^{(0)} \in \mathbb{R}^{n \times k}$ the initial orthonormal frame ($Q^{(0)\top} Q^{(0)} = I_k$). Each step performs a thin QR factorisation and feeds the resulting frame to the next step,

$$J_t Q^{(t)} = Q^{(t+1)} R^{(t+1)}, \quad t = 0, \dots, N-1, \quad (11)$$

with $Q^{(t+1)}$ orthonormal-columned and $R^{(t+1)}$ upper triangular with positive diagonal. Substituting (11) repeatedly telescopes the rollout Jacobian $M_N = \prod_{t < N} J_t$ acting on the initial frame,

$$M_N Q^{(0)} = J_{N-1} \cdots J_0 Q^{(0)} = Q^{(N)} \underbrace{R^{(N)} R^{(N-1)} \cdots R^{(1)}}_{\mathcal{R}_N}. \quad (12)$$

Equation (12) is itself a thin QR factorisation of $M_N Q^{(0)}$: the intermediate frames are not cancelled but carried forward as each step’s input, and the entire accumulated rotation collects in the single factor $Q^{(N)}$. Two properties then isolate the spectrum.

First, $\mathcal{R}_N$ is a product of upper-triangular matrices, hence upper triangular with diagonal equal to the product of diagonals,

$$(\mathcal{R}_N)_{ii} = \prod_{t=1}^N R_{ii}^{(t)} \quad \implies \quad \ln(\mathcal{R}_N)_{ii} = \sum_{t=1}^N \ln R_{ii}^{(t)}, \quad (13)$$

This replaces the overflow-prone product with an additive running sum, explaining why we re-orthonormalise at every step instead of forming M_N once.

Second, $Q^{(N)}$ has orthonormal columns and is therefore an isometry: $\|Q^{(N)}x\| = \|x\|$, its singular values are all one, $\|Q^{(N)}\| = 1$, and so $\sigma_i(M_N Q^{(0)}) = \sigma_i(\mathcal{R}_N)$. It rotates the frame but neither stretches nor shrinks it, contributing nothing to any growth rate. Dividing (13) by the elapsed physical time $N dt$ and letting $N \rightarrow \infty$ gives

$$\lambda_i = \lim_{N \rightarrow \infty} \frac{1}{N dt} \sum_{t=1}^N \ln R_{ii}^{(t)}, \quad (14)$$

with convergence guaranteed by Oseledets’ theorem for a generic $Q^{(0)}$. “Generic” is what the seeded *random* frame of Section 3.2 secures: with probability one the columns of $Q^{(0)}$ avoid the measure-zero subspaces of the Oseledets filtration that grow more slowly than the top- k rates, so (14) returns $\lambda_1 \geq \dots \geq \lambda_k$ rather than a slower set. That the *diagonal* of $\mathcal{R}_N$ suffices—rather than its singular values—is exactly because the isometry $Q^{(N)}$ ties the two together.

References

- [1] Valery I. Oseledets. “A Multiplicative Ergodic Theorem. Lyapunov Characteristic Numbers for Dynamical Systems”. In: *Transactions of the Moscow Mathematical Society* 19 (1968), pp. 197–231.
- [2] Jean-Pierre Eckmann and David Ruelle. “Ergodic Theory of Chaos and Strange Attractors”. In: *Reviews of Modern Physics* 57.3 (1985), pp. 617–656.
- [3] Giancarlo Benettin et al. “Lyapunov Characteristic Exponents for Smooth Dynamical Systems and for Hamiltonian Systems; a Method for Computing All of Them”. In: *Meccanica* 15 (1980), pp. 9–30.
- [4] Rainer Engelken, Fred Wolf, and L. F. Abbott. “Lyapunov Spectra of Chaotic Recurrent Neural Networks”. In: *Physical Review Research* 5 (2023), p. 043044.
- [5] Miles Macklin. *Warp: A High-performance Python Framework for GPU Simulation and Graphics*. NVIDIA GPU Technology Conference (GTC). 2022. URL: <https://github.com/NVIDIA/warp>.
- [6] Hyung Ju Terry Suh et al. “Do Differentiable Simulators Give Better Policy Gradients?” In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR. 2022.
- [7] Paavo Parmas et al. “PIPPS: Flexible Model-Based Policy Search Robust to the Curse of Chaos”. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR. 2018.
- [8] Luke Metz et al. “Gradients Are Not All You Need”. In: *arXiv preprint arXiv:2111.05803* (2021).
- [9] Vladimir I. Arnold. *Mathematical Methods of Classical Mechanics*. 2nd ed. Vol. 60. Graduate Texts in Mathematics. Springer, 1989.
- [10] Russ Tedrake. *Underactuated Robotics: Algorithms for Walking, Running, Swimming, Flying, and Manipulation*. Course Notes for MIT 6.832. 2023. URL: <https://underactuated.mit.edu>.
- [11] Mark W. Spong. “The Swing Up Control Problem for the Acrobot”. In: *IEEE Control Systems Magazine* 15.1 (1995), pp. 49–55.
- [12] Michel Hénon and Carl Heiles. “The applicability of the third integral of motion: Some numerical experiments”. In: *The Astronomical Journal* 69 (1964), pp. 73–79.
- [13] Ernst Hairer, Christian Lubich, and Gerhard Wanner. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. 2nd ed. Vol. 31. Springer Series in Computational Mathematics. Springer, 2006.

- [14] Benedict Leimkuhler and Sebastian Reich. *Simulating Hamiltonian Dynamics*. Cambridge Monographs on Applied and Computational Mathematics. Cambridge University Press, 2004.
- [15] Sam Greydanus, Misko Dzamba, and Jason Yosinski. “Hamiltonian Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.
- [16] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. “Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems”. In: *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13.5 (1983), pp. 834–846.